T06b Enemy Attack

CRE132 worksheet

What you'll make

T06's slimes get teeth. Walk close and a slime chases you; when it reaches you it jumps on you, once every 1.2 seconds, and you go redder. Run far enough away and it gives up and hops all the way home before it will chase you again.

You won't start a new script: you'll add two states, **Attack** and **ReturnHome**, to T06's `EnemyAI`. By the end you'll know the three places a new state has to go (the enum, a method of its own, and a `case` in each `switch`), and why a state that ignores the player for a while makes an enemy behave better.



Figure 1: The finished meadow: the player on a patch of dirt, the edge of the pond at the top and a row of bushes below.

Before you start

1. Unzip `CRE132_T06b_EnemyAttack_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T06b_EnemyAttack_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T06b_EnemyAttack`.
3. Press **Play**. This is T06's meadow, and the slimes already have T06's **EnemyAI**: they idle, wander and chase you, and give up when you run away, but they can't hurt you. Press Play again to stop.

You'll change `Assets/_game/Scripts/T06_EnemyAI/EnemyAI.cs`. Open it now. If you'd rather carry on with your own T06 script, copy it over this one first.

1. Two more states

Add two boxes and their arrows to your T06 drawing:

State	What it does	When it leaves
Chase (changed)	runs at the player	within 0.9: **Attack**; player beyond 7: **ReturnHome** (not Idle)
Attack	jumps on the player	after 1.2 seconds: back to **Chase**
ReturnHome	walks back home, ignoring the player	got home: **Idle**

Every new state needs three things: a new name in the **enum**, a new **`UpdateX` method**, and a new **`case`** in **both** `switch` statements (in `Update` and in `ChangeState`). Forgetting a `case` is the usual mistake: the slime enters the state and then does nothing.

2. The enum and the settings

If you like, replace the comment at the very top of the file so it describes the new states:

```
// A simple enemy "brain" built as a state machine. The enemy is always in exactly one
// state, and each state has its own rules for what to do and when to switch to another
// state:
//   Idle       wait a moment, then Wander (or Chase if the player comes close)
//   Wander     walk to a random spot near home, then Idle (or Chase if the player comes
//              close)
//   Chase      run at the player; Attack when close enough, ReturnHome if the player gets
//              away
//   Attack     jump on the player, then go back to Chase
//   ReturnHome walk back home, then Idle
// It moves in straight lines, so trees and water get in its way: T07 fixes that with
// pathfinding.
```

Add **Attack** and **ReturnHome** to the enum. Note the comma after **Chase** now:

```
    private enum State
    {
        Idle,
        Wander,
        Chase,
        Attack,
        ReturnHome
    }
```

Below the `giveUpRange` line, add the attack settings:

```
    [Header("Attack")]
    [SerializeField] private float attackRange = 0.9f;
    [SerializeField] private int attackDamage = 1;
    // When the hit lands, in seconds from the start of the attack (as the slime comes
    // down from its jump).
    [SerializeField] private float hitTime = 0.5f;
    // How long one attack takes, including a short rest after the hit.
    [SerializeField] private float attackDuration = 1.2f;
```

And below `private float idleTime;`, add:

```
    private bool hasHit;
```

- **`hitTime` and `attackDuration` are timers**, measured from the start of the attack: the hit lands at 0.5 seconds (as the slime comes down from its jump) and the whole attack takes 1.2.
- **`hasHit`** remembers whether this attack has already landed, so it hits only once.

3. Two new cases in each switch

In `Update`'s `switch`, after the **Chase** case's `break;`, add:

```
            case State.Attack:
                UpdateAttack();
                break;
            case State.ReturnHome:
                UpdateReturnHome();
                break;
```

In `ChangeState`'s `switch`, after the **Chase** case's `break;`, add:

```
            case State.Attack:
                hasHit = false;
                PlayAnimation("Slime_Attack");
                break;
            case State.ReturnHome:
                moveSpeed = wanderSpeed;
                PlayAnimation("Slime_Move");
                break;
```

- **Attack's start-of-state job** resets `hasHit` and plays **Slime_Attack**. It sets no speed, and `ChangeState` has already stopped the slime, so it jumps on the spot.
- **ReturnHome walks at `wanderSpeed`**, like Wander.

4. Chase: give up to ReturnHome, attack when close

In `UpdateChase`, there are two changes. In the `player == null` check at the top, change `ChangeState(State.Idle);` to:

```
            ChangeState(State.ReturnHome);
```

Then change the distance check so it reads:

```
        if (distance > giveUpRange)
        {
            ChangeState(State.ReturnHome);
        }
        else if (distance <= attackRange)
        {
            ChangeState(State.Attack);
        }
```

The `else` with `MoveTowards(player.position);` stays below it.

- **Too far: go home. Close enough: attack. Otherwise: keep running at the player.**

5. The Attack and ReturnHome methods

Below `UpdateChase`, add:

```
    private void UpdateAttack()
    {
        // The hit lands once, partway through. If the player has stepped away by then, it
        // misses.
        if (!hasHit && stateTimer >= hitTime)
        {
            hasHit = true;
            if (player != null &&
                Vector2.Distance(transform.position, player.position) <= attackRange)
            {
                player.GetComponent<Health>().TakeDamage(attackDamage);
            }
        }

        if (stateTimer >= attackDuration)
        {
            // Chase decides what's next: attack again, keep chasing or give up.
            ChangeState(State.Chase);
        }
    }

    private void UpdateReturnHome()
    {
        // This state ignores the player, so the enemy really does go home before it can
        // chase again.
        if (HasReached(homePosition))
        {
            ChangeState(State.Idle);
        }
        else
        {
            MoveTowards(homePosition);
        }
    }
```

- **A timer instead of Animation Events.** The hit lands at `hitTime`, but only if the player is still within range. Step away in time and it misses.
- **`hasHit = true` comes first**, so even a miss uses up this attack's one hit.
- **After `attackDuration` it goes back to Chase**, which decides what's next: attack again, keep chasing or give up.
- **ReturnHome ignores the player**, so the slime really does go home before it can chase again.

6. The red gizmo

At the end of `OnDrawGizmosSelected`, after the grey give-up range, add:

```
        Gizmos.color = Color.red;
        Gizmos.DrawWireSphere(transform.position, attackRange);
```

Save. The Console should be clear of red errors.

Add it: there's nothing to add: the slimes already have **Enemy AI**. Select a slime: the new **Attack** settings have appeared with the values saved in the scene: **Attack Range** 0.9, **Attack Damage** 1, **Hit Time** 0.5, **Attack Duration** 1.2. Leave them as they are.

Test it: press Play and walk up to **Slime 1**. It chases you, then jumps on you once every 1.2 seconds, and you go redder. Step away just as it jumps and it misses. Run far away: it hops all the way home before it will chase you again. Watch **Current State** in the Inspector and the Console as it changes.

7. The problem for next time

Walk to the bottom of the pond so that **Slime 2** is on the other side, and let it notice you. It runs straight at you, into the water's edge, and stays stuck. It only knows "move towards", not how to get round things. T07 adds pathfinding.

Check your work

- ☐ Within 0.9 units a slime attacks, once every 1.2 seconds, and you go redder.
- ☐ Stepping away just as it jumps makes it miss.
- ☐ Running more than 7 units away sends it home, and it ignores you until it gets there.
- ☐ The red attack range shows when a slime is selected.
- ☐ Compare with `CRE132_T06b_EnemyAttack_Finished` if something behaves differently.

Key points

- **Adding a state** means a new enum value, a new `UpdateX` method, and a new `case` in each `switch`.
- **A state that ignores the player for a while** (ReturnHome) stops the enemy getting stuck in a chase loop.
- **A timer can stand in for Animation Events** when the timing is simple.

If something goes wrong

- **The slime reaches you and then just sits there:** the **Attack** case is missing from `Update`'s `switch`, so `UpdateAttack` never runs.
- **The slime gives up and then stands still for ever:** the **ReturnHome** case is missing from `Update`'s `switch`.
- **The slime hurts you once and never again, or plays the wrong animation:** a case is missing from `ChangeState`'s `switch` (it resets `hasHit` and picks the animation).
- **"The name 'attackRange' does not exist":** the attack settings from part 2 are missing, or are outside the class.
- **NullReferenceException in UpdateAttack:** the **Player** has no **Health** component.
- **The Attack settings all show 0:** you typed the field names differently from the listing, so Unity couldn't match the saved values. Fix the names, or type the values in.

Try it

1. Add a **Flee** state: run away from the player when the slime's health is 1.
2. Give the slime a **Hurt** state that pauses it for half a second when hit. (Hint: `Health` has an **On Damaged** event.)